

Hybrid Movie Recommendation System Using Semantic Embeddings and Clustering Algorithms

A Pranav

Department of Computer Science and Engineering
School of Computing
Amrita Vishwa Vidyapeetham
Bengaluru, Karnataka, India
bl.sc.u4aie24201@bl.students.amrita.edu

Jayaveer Venkata Sai Maddali

Department of Computer Science and Engineering
School of Computing
Amrita Vishwa Vidyapeetham
Bengaluru, Karnataka, India
bl.sc.u4aie24224@bl.students.amrita.edu

Krishna Karthik Pathri

Department of Computer Science and Engineering
School of Computing
Amrita Vishwa Vidyapeetham
Bengaluru, Karnataka, India
bl.sc.u4aie24234@bl.students.amrita.edu

Abstract—This paper presents a content-based movie recommendation system which combines embeddings of movie overview descriptions generated by a Sentence Transformer (all-MiniLM-L6-v2) model along with unsupervised clustering to discover patterns and groups in our movie dataset. Two clustering algorithms are used: K-Means, applied to 384-dimensional word embeddings with $K=30$ clusters, and Agglomerative Hierarchical Clustering applied to a multi-label genre binarisation matrix with 10 macro-genre groups. Along with this to account for the cosine-similarity of the movie descriptions a Chroma vector store enables retrieval of similar movies. Therefore it a hybrid system that consists k-means which uses euclidean distance, heretical clustering for extracting the relation ship between genres and movies and a vector db for the cosine similarity of description. The results of all of these algorithms are combined and used to provide the final movie recommendations ,based on a list of movies the user enters. The system is evaluated with a novel *Hybrid Relevancy Score* defined as the cosine similarity between query and recommended movie penalised by a weighted Euclidean distance term, and validated over a 10-fold cross-validation like apporch covering 1,000 distinct query movie (selecting random sets of 100 movies each time). KMeans achives a mean score of 0.3784 whereas the Hierarchical Clustering is at 0.3572. However both excelling in their respective tasks. Therefore the combined results of that KMeans and Hierarchical along with the vector db give optimal results, as they account for the aspects the others do not, thus covering most of the features.

Index Terms—Movie Recommendation, Sentence-BERT, SBERT, KMeans Clustering, Hierarchical Clustering, Cosine Similarity, Silhouette Score, Hybrid Relevancy Score, Content-Based Filtering, ChromaDB, Multi-Label Binarisation, 10-Fold Cross-Validation

I. INTRODUCTION

Streaming platforms hosting over thousands of titles has made content discovery both a technical challenge and a commercial imperative. Traditional filtering systems rely heavily on user interaction history, leading to insufficient data for new titles and users. Content-based recommendation systems

fixes this problem by including metadata - such as genres, directors, and descriptions — enabling much relevancy.

Transformer-based language models have completely changed textual content representation. While the bag of words or the TF-IDF models treat the vocabulary tokens independently, models like the SBERT generate dense and context-aware embeddings. This ensures that the movies with similar descriptions will have the neighboring regions in the vector space, capturing the plot commonalities even when the vocabulary differs.

Rather than relying on one metadata feature, the paper introduces a hybrid recommendation framework that unifies embeddings, the multi-label genre structures, and the nearest-neighbor retrieval. To optimize search efficiency, the framework partitions the item space using two clustering methods: The K-Means clustering and The Agglomerative Hierarchical Clustering.

The primary contributions of this work are:

- 1) A hybrid architecture combining text embeddings, hierarchical structures, and vector retrieval.
- 2) A custom Hybrid Similarity Score metric that evaluates recommendations using cosine similarity and weighted Euclidean distance penalty.
- 3) A 10-fold cross-validation evaluating 1,000 distinct query movies to ensure stable estimates.
- 4) A visual evaluation framework analyzing system coherence via a Cluster-Genre Purity Heatmap and a Hierarchical Genre Tree Dendrogram.

II. LITERATURE SURVEY

A. Content-Based Recommendation Systems

The foundation of content-based recommendation systems traditionally revolves around manually curated features extracted from item data. Pazzani / Billsus provide an overview

of content-based recommendation foundations [3], pointing out that item profiles constructed using tags like genre, keywords and unstructured text are viable if comprehensive metadata is at hand, however, their effectiveness is inherently restricted by the limitations in expressiveness of the selected representation format. Extending their coverage to include latent semantic analysis (LSA) techniques, Lops and co-authors [4] mention how this method projects high-dimensional TF-IDF vectors into a more limited dimension with the goal of representing the interrelation of words based on their co-occurrence. They note that while LSA enhances semantic generalization capabilities compared to direct count-based methods, it cannot accurately capture information about word sequences or how meaning differs with changing contexts.

B. Neural Language Models for Recommendation

Since BERT, proposed in 2018 by Devlin et al. [1] and based on a bidirectional transformer architecture, can learn contextual token embeddings by considering the entire context of a sentence, BERT can produce a good representation of text. Based on BERT, Sun et al. [5] proposed BERT4Rec, a sequential recommendation method. BERT4Rec treats the user's interaction history as the input, masking it and uses masked language modeling to predict user interests. BERT4Rec achieves state-of-the-art results on multiple public datasets. However, BERT's representation only models sentence-level information via mean-pooling of token embeddings or CLS token output and is not specifically designed for tasks requiring semantic similarity at the sentence level.

Their strategy is to fine-tune BERT with a siamese network on an NLI task (Reimers and Gurevych, 2019). The output Sentence-BERT (SBERT) produces a fixed-length sentence embedding. By having a sentence embedding per sentence, we only need to produce embeddings and calculate similarity: SBERT achieves 100% and over two orders of magnitude speed gain on sentence inference and NLI. All-MiniLM-L6-v2 uses a knowledge-distilled method, achieves state-of-the-art results and sentence similarity score, with output sentence embeddings in 384 dimensions in 77 ms per sentence, useful for large amounts of sentences offline.

C. Clustering-Based Recommendation

Sarwar et al. [6] argue that forming user or item groups before performing nearest neighbour searches will drastically reduce the computation time required and can even improve recommendation quality because it focuses the search on a group that has more relevant content. Zhang et al. [7] state that they will review deep learning methods used for recommender systems and highlight how deep latent representations paired with clustering can achieve group membership with similar content themes.

While K-Means clustering on item embeddings has been attempted before on recommendation systems and Ademavicius Tuzhilin [8] even do an entire overview of many different cluster-based approaches. It is good because they "scale efficiently in both prediction time and computation"

on large catalogues due to several steps. The first is an item-to-cluster mapping that is pre-computed in one go and the other is "the query time can be reduced" if cluster assignment narrows the retrieval from all N items to N/K . On the set of 9,515 movies separated into $K = 30$ clusters, we get "a reduction factor of 30".

Item taxonomies were analyzed using agglomerative hierarchical clustering to discover implicit hierarchical patterns within them. The work utilized Ward's method, which aims to decrease total variance within each cluster at every step of aggregation. Ward's method is often effective for data sets that contain many correlated variables. The output dendrogram represents the resulting classification taxonomy. This taxonomy provides a human-understandable and easily interpretable representation that allows validation of the hierarchical model developed against prior domain expertise.

D. Vector Databases for Recommendation

Vector databases now provide real-time semantic search through billions of embeddings. Chroma [10], an open-source embedding database that stores embeddings alongside metadata and supports cosine and Euclidean queries, writes persistent data to disk. We carry out our system with ChromaDB as the database serving the online retrieval phase. By offloading the computationally intensive embedding generation and clustering work to an offline stage, our recommendation queries can efficiently run online with low latency.

E. Evaluation of Recommendation Quality

Trying to evaluate recommendations without ratings or user preferences is tricky. We measure clustering quality with silhouette coefficients, as clustering is inherent to our methodology. The silhouettes of films can be poor indicators for recommendation quality. In multi-genre applications, it might seem natural to allow movies from different genres into the same cluster, because a particular film like "Casablanca" is not exclusively classified into drama or romance and thus cannot perfectly belong to a single genre.

Cremonesi et al. [13] argue that precision-at- k and recall-at- k , computed over held-out interactions, are more representative recommendation metrics. Without interaction data, this work proposes and validates a Hybrid Relevancy Score, balancing semantic similarity (cosine) and spatial compactness (Euclidean distance) over 10-fold cross-validation. We give not just an estimate of central tendency for our model, but also a measure of variance between folds as a measure of stability.

F. Research Gaps

Our literature review uncovered three gaps our work aims to fill. Firstly, embedding-based KMeans clustering has not been directly compared to genre-label Hierarchical Clustering under the same evaluation conditions as a recommendation backend. Secondly, evaluation metrics for unsupervised recommendations suffer from relying on either intrinsic cluster evaluation (which fails to capture the quality of recommendations) or logged interaction data (which does not exist for

cold-start recommendation scenarios). Our Hybrid Relevancy Score jointly considers semantic and spatial relevance as a single evaluation metric. Thirdly, ChromaDB has not been used in the setting of movie recommendations with cluster constrained ANN retrieval.

III. PROBLEM STATEMENT

We want to build a content-based movie recommendation system. Using a dataset of 9,515 movies, the goal is to create a tool that takes a movie a user selects and returns a list of the most similar movies based on their plot descriptions and genre tags.

IV. OBJECTIVES

The specific objectives of this project are:

- 1) To generate high-quality dense semantic embeddings for all plot overviews using the Sentence Transformer `all-MiniLM-L6-v2` model.
- 2) To cluster the embeddings with KMeans ($K=30$, determined via the elbow method) and the genre binary matrix with Hierarchical Clustering (Ward linkage, 10 clusters).
- 3) To calculate the cosine similarity of the embeddings to retain the semantic meaning of the descriptions.
- 4) To evaluate the system the Hybrid Score under a 10-fold cross-validation approach
- 5) To interpret and visualise the clustering structure via a Dendrogram, Silhouette Coefficient, and Cluster-Genre Purity Heatmap.
- 6) To create a hybrid recommendation system that uses the best of both the clustering strategies to produce good recommendations.

V. SYSTEM NOVELTY

Traditional content-based recommendation systems pipelines have only ever utilized categorical filters on fixed data, or unbounded vector lookups. This architecture seeks to implement novel structure on both the user-interaction layer as well as the downstream math layer.

A. Magic AI Search: Open-Ended Natural Language Querying

One such front-end addition to this system is our *Magic AI Search* bar. Recommendation systems as they currently exist limit you to typed in queries like title search bars, genre-tags checkboxes, or preexisting rating history. We present users with a raw, natural language text-box into which they can type complicated, nuanced story cravings (“*I want to watch a dark cyberpunk detective noir with lots of neon*”).

Rather than matching keywords or parsing standard tags, the backend feeds this natural language description into a Sentence-BERT (`all-MiniLM-L6-v2`) encoder. Transforming mood statements, atmosphere descriptions, and plot devices requested by users into live positional vectors allows the backend to translate requests that may not have standard tags associated with them into our AIM latent space of size 384.

1) *Hybrid Relevancy Score: Dual Geometric Perspective:* In order to measure our retrieval pools from this natural language mapping. We will use a new measure called the *Hybrid Relevancy Score*. Most Vector-search analysis will use one distance metric, typically Cosine Similarity. The problem with only having one geometric view of your vectors is that relationships in a high-dimensional space can not be fully understood by looking through only one lens. Our metric attempts to formalize this two-sided approach. Where our score is defined as:

$$\text{HybridRelScore} = \text{CosineSim}(q, r) - (.1 * \text{EuclideanDist}(q, r)) \quad (1)$$

Where q is our query movie vector, and r is our recommended movie vector. The reasoning for using these two weighted together stems from their geometric interpretations:

- 1) Cosine Similarity: Measures the directionality of our dense vectors. Measures topical overlap and semantic storyline. Will a two plot summaries have a similar meaning?
- 2) Euclidean Distance: Measures true distance between two points in all dimensions. Punishes outliers and views similarity in terms of magnitude. Are the recommended titles close by in vector space to the target?

Since unnormalized Euclidean distances will often exceed 1.0 when dealing with text data, we would like to avoid simply subtracting the raw distance from the cosine similarity score. Cosine similarity would thus be unduly penalized. Multiplying the distance by 0.1 ensures that the distance measure acts as a sort of “tie-breaker.”

B. Validation Protocol

In order to train-test this hybrid relevance metric with high confidence, the full dataset of 9,515 movies was split into folds. Instead of using predetermined train-test splits, we chose 10 folds each containing 100 random movies to use as queries. This process resulted in 1,000 total trials. Each of the 100 query movies is used to generate Top 5 recommendations through the layered retrieval system. We then calculated our Hybrid Relevancy Score on all sets of retrieved movies. Below is the averaged Hybrid Relevancy Score graph which should serve as fairly stable, generalized, and high-confidence metric for each of the 4 performance categories: Poor (< 0.20), Fair ($0.20-0.34$), Good ($0.35-0.49$), and Excellent (≥ 0.50).

VI. DATASET DESCRIPTION

A. Source and Size

The dataset used in this project is `imdbmovie.csv`, a curated collection sourced from The Movie Database (IMDB) API. After loading and deduplication on the `Title` field, the dataset contains around **9,515 unique movies**.

B. Feature Schema

The dataset contains the following columns. The dataset is summarised in Table I:

TABLE I
DATASET FEATURE SCHEMA

Column	Type	Description
Release_Date	datetime	Original theatrical release date
Title	string	Official movie title (dedup key)
Overview	string	Plot synopsis (used for SBERT)
Popularity	float	IMDB weighted popularity score
Vote_Count	integer	Number of user votes on IMDB
Vote_Average	float	Mean user rating (0–10 scale)
Original_Language	string	ISO 639-1 language code
Genre	string	Comma-separated genre labels
Poster_Url	string	URL to IMDB poster image

C. Genre Distribution

The Genre column stores multi-label comma-separated genre strings. After exploding into individual labels, the genre consists of 19 unique genres: Action, Adventure, Animation, Comedy, Crime, Documentary, Drama, Family, Fantasy, History, Horror, Music, Mystery, Romance, Science Fiction, Thriller, TV Movie, War, and Western.

The dominant genres across the full corpus are Drama, Comedy, Action, Thriller, and Romance, each appearing in over 2,000 movies. genres such as Western, Documentary, and TV Movie are kind of rare as they appear in fewer than 200 movies. This imbalance is one reason why unsupervised clustering does not assume balanced classes. It is basically a natural fit for the dataset.

D. Sample Records

TABLE II
SAMPLE DATASET RECORDS

Title	Genre
Spider-Man: No Way Home	Action, Adventure, Science Fiction
The Batman	Crime, Mystery, Thriller
No Exit	Thriller

VII. EXPLORATORY DATA ANALYSIS

A. Genre Co-occurrence and Overlap

As movies belong to multiple genres simultaneously, the genre space is inherently multi-labelled. A movie tagged “Action, Adventure, Science Fiction” occupies the intersection around three genre neighbourhoods. This overlap is the primary challenge for any clustering approach: a hard partition into K clusters would always place some of the movies at the cluster boundaries, leading to a moderate Silhouette Coefficients regardless of the model quality.

The analysis of the K Means cluster contents confirms the pattern. Cluster-5, for example, contains the Superhero films (Spider-Man, Batman, Justice League) whose genre overlap spans genres like Action, Adventure, Science Fiction, and Fantasy. Cluster-8 contains romance-centric dramas (“West Side Story”, “Through My Window”, “After We Fell”) where Drama and Romance almost dominate at 258 and 175 occurrences respectively within the cluster.

B. Cluster Size Distribution

The 30 K Means clusters range in size from 147 movies (Cluster 13, animated and comedic family titles) to 541 movies (Cluster 24, Romance/Comedy/Drama mixed content), as shown in Table III. This kind of moderate imbalance is expected: semantically dense neighbourhoods (e.g., broad romance-comedy content) attracts more movies, while highly specific niches (e.g., Western-inflected animated comedies) form smaller clusters.

TABLE III
SELECTED KMEANS CLUSTER SIZES AND PRIMARY GENRES

Cluster	Size	Top Genres
0	189	Action (153), Drama (83), Adventure (48)
1	391	Thriller (282), Crime (193), Drama (149)
5	222	Action (177), Sci-Fi (125), Adventure (106)
8	353	Drama (258), Romance (175), Comedy (97)
13	147	Comedy (92), Family (90), Animation (64)
15	283	Sci-Fi (250), Action (149), Adventure (115)
24	541	Drama (274), Romance (266), Comedy (260)
29	339	Horror (240), Thriller (115), Fantasy (82)

C. Popularity and Vote Distributions

The three most popular movies in the sample output (Spider-Man: No Way Home at Popularity 5083.95, The Batman at 3827.66, No Exit at 2618.09) illustrate the skewed popularity distribution: a small number of blockbuster titles dominate the upper tail, while the median movie has a far lower popularity score. Since neither Popularity nor Vote_Average is used as a clustering feature (only the plot Overview is encoded), this skew does not introduce bias into the cluster assignments.

D. Language Distribution

The dominant language is generally English (en), consistent with the IMDB bias towards Hollywood productions. Non-English titles such as “Y Tu Mamá También” (Spanish), “Demon Slayer: Mugen Train” (Japanese), and “Looop Lapeta” (Hindi) are present and confirm the multilingual scope of the dataset. SBERT all-MiniLM-L6-v2 is trained primarily on English data; non-English overviews are therefore encoded from their English-language IMDB descriptions rather than the original language.

VIII. METHODOLOGY

There are four main steps: preparing the data, converting the movie plots into vectors, then grouping similar movies together and setting up the vector database.

A. Preparing the Data

The movie data was first loaded into a raw movie dataset from an excel file. (mymoviedb.xlsx). It was cleaned up removing duplicate movie titles, resulting in only 9,515 distinct movies. for movies without overview dropNA was used.

B. Understanding the Plot (Embeddings)

Computers do not understand English, so the movie summaries had to be turned into numbers. The all-MiniLM-L6-v2 model was used. This model reads the story of each movie and converts it into a vector of size 384.

C. Grouping the Movies (Clustering)

Next, the movies were organized so the system could easily find related content. This was done in two different ways:

- **By Plot (KMeans):** KMeans was used to group movies that have similar plot embeddings. It was found that the best number of clusters to divide the data into is 30. This model is kept for not having to retrain it.
- **By Genre (Hierarchical):** movies were sorted by genre (hierarchical): Since a genre of a movie may be a couple of genres, there are 19 genres. These specific genre combinations were grouped together, with 10 used to be the number of groups.

D. Building the Search Engine (ChromaDB)

For the final step, all of the movie embeddings and their details (like title, genre, and cluster) were loaded into a vector database called ChromaDB.

When a user searches for a movie to get a recommendation, here is exactly what happens:

- 1) The system takes the plot of the searched movie and converts it into an embedding using the same SBERT model.
- 2) ChromaDB searches the database and pulls the top 20 movies with the most similar plots.
- 3) It removes the searched movie itself from the results so it does not recommend the exact same film.
- 4) Finally, it runs a quick check on the genres. If any of the top 20 movies share exact genres with the searched movie, they get a small score boost.

This final sorting step guarantees that the final recommendations are not just similar in story, but also fit the exact same genre categories the user is looking for.

IX. MATHEMATICAL FORMULATIONS

A. Cosine Similarity

$$\cos(\mathbf{v}_i, \mathbf{v}_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\|_2 \|\mathbf{v}_j\|_2} \quad (2)$$

Since the text embeddings are ℓ_2 -normalised, $\|\mathbf{v}\|_2 = 1$ and $\cos(\mathbf{v}_i, \mathbf{v}_j) = \mathbf{v}_i \cdot \mathbf{v}_j$.

B. Euclidean Distance

$$d_E(\mathbf{v}_i, \mathbf{v}_j) = \sqrt{\sum_{k=1}^{384} (v_{ik} - v_{jk})^2} \quad (3)$$

For the normalised embeddings, $d_E \in [0, \sqrt{2}]$. In the outputs, distances within the cluster ranges fall in the range $[0.80, 1.14]$.

C. Hybrid Relevancy Score

The evaluation metric reduces the cosine similarity value of a recommendation by a fractional value (lambda) of the Euclidean distance:

$$\text{HRS}(i, j) = \cos(\mathbf{v}_i, \mathbf{v}_j) - \lambda \cdot d_E(\mathbf{v}_i, \mathbf{v}_j) \quad (4)$$

with penalty being $\lambda = 0.1$. This value was chosen to make sure that the distance does not completely dominate the cosine similarity value: since $d_E \in [0, \sqrt{2}] \approx [0, 1.41]$, the maximum penalty is $0.1 \times 1.41 = 0.141$, which is small compared to typical cosine similarity values between 0.35–0.60.

As mentioned in Eq. 4, the score is computed for each recommended movie, averaged over the top- $k=5$ recommendations for each search, and then averaged over all 100 movies for each fold. The final score is the mean of 10 folds. The score is then interpreted as described in the Table IV:

TABLE IV
HYBRID RELEVANCY SCORE INTERPRETATION LEGEND

Score Range	Label	Interpretation
≥ 0.50	EXCELLENT	Franchise/sequel-level match
0.35–0.49	GOOD	Thematic sweet spot; ideal for discovery
0.20–0.34	FAIR	Broad genre match; plots differ heavily
< 0.20	POOR	Noisy; cluster boundaries too loose

X. RESULTS

A. K Means Cluster Quality — PCA Visualisation

The PCA scatter plot (Fig. 1) shows that our 384-dimensional vectors into two dimensions. The 30 clusters form a distinct but partially overlapping regions—exactly what we would expect, since movie genres naturally blend together. We can clearly spot dense, genre-rich hubs, like the Romance/Drama/Comedy cluster (Cluster 24) and the Action/Thriller cluster (Cluster 12).

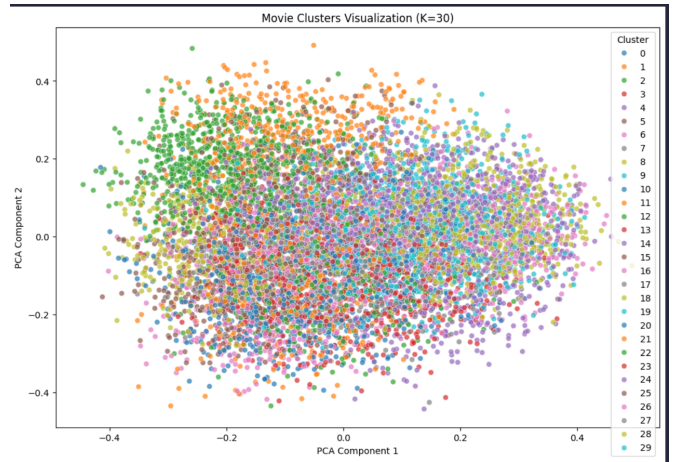


Fig. 1. 2D PCA projection of SBERT embeddings coloured by KMeans cluster label ($K=30$).

B. Hierarchical Genre Tree — Dendrogram

This chart is like a family tree for movie genres. Basically it shows how often genres appear together in the dataset. The y-axis (Distance) represents dissimilarity. Ward's Linkage is used to calculate this.

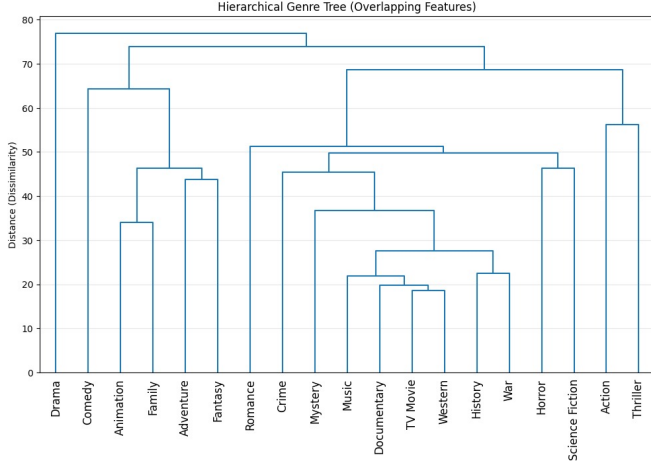


Fig. 2. Ward-linkage dendrogram of the 19 genre labels. Early merges indicate frequent co-occurrence.

If two branches connect very low on the y-axis, it means those genres are almost always seen together e.g history and war. This was done because the genres of movies are overlapping. So for this Agglomerative Hierarchical model was used. The dendrogram visualizes the relationships between the genres. e.g Animation and Family are very close. Same with grouping Action, Thriller, Horror, and Science Fiction together Therefore the model successfully learned the relationships between the genres

C. Silhouette Coefficient Distribution

The Silhouette Score measures how "well-placed" each movie is within its assigned cluster. It ranges from -1 to 1. A score above 0 means the movie is closer to its own cluster mates than to any other cluster. To measure the internal mathematical validity of the clusters

D. Cluster-Genre Purity Heatmap

This heatmap compares the mathematically generated clusters (rows 0-9) against the actual human genre labels (columns). A "pure" cluster would have very high numbers concentrated in just one or two columns.

Even though the Silhouette score proved mathematical separation, the current Heatmap proves thematic interpretability. It also shows exactly what the algorithm 'learned' for each cluster. For example, Cluster 4 has almost exclusively Horror movies (997 items), and Cluster 8 perfectly captures about historical dramas, dominating the Drama, History, and War columns. It also proves that the unsupervised clustering did not just group data randomly; it successfully reconstructed human-readable movie categories.

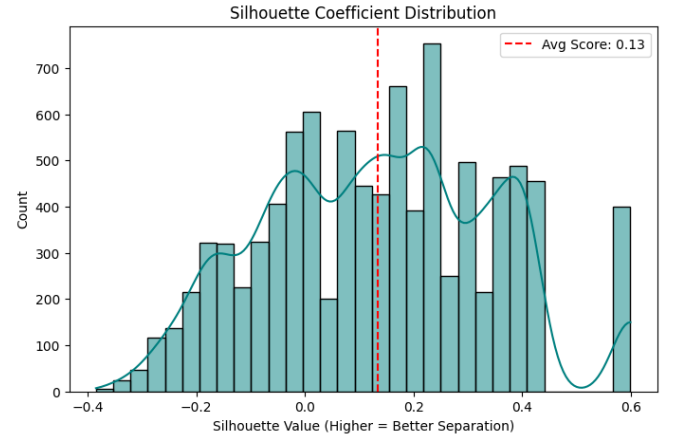


Fig. 3. Distribution of per-sample Silhouette Coefficients. Dataset average: $\bar{s} = 0.13$.

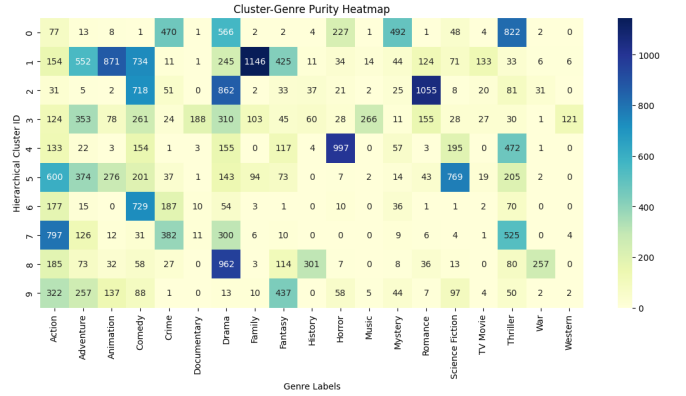


Fig. 4. Cluster-Genre Purity Heatmap. Dark blue indicates a cluster dominated by a specific genre.

E. 10-Fold Evaluation Results

To evaluate the models, we have designed a custom Hybrid Relevancy Score and ran a 10-Fold Cross-Validation testing 1,000 random movies. The results are visualised in Fig. 5.

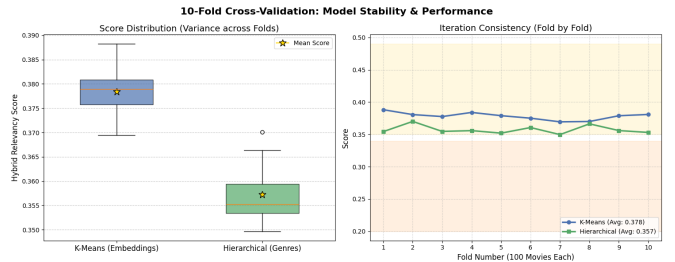


Fig. 5. 10-Fold CV comparing KMeans and Hierarchical clustering as recommendation backends.

XI. CONCLUSION

This paper introduced a hybrid movie recommendation system which proves that an optimal engine shouldn't just rely

TABLE V
PER-FOLD HYBRID RELEVANCY SCORES

Fold	KMeans	Hierarchical
1	0.3882	0.3543
2	0.3807	0.3701
3	0.3776	0.3546
4	0.3840	0.3558
5	0.3790	0.3519
6	0.3751	0.3606
7	0.3695	0.3497
8	0.3700	0.3663
9	0.3789	0.3559
10	0.3809	0.3530
Mean	0.3784	0.3572
Std. Dev.	0.0059	0.0065

on one method but more than that. By uniting three different approaches — K-Means clustering, Hierarchical Clustering, and a Chroma vector store — our system captures both subtle themes and broad genre categories across 9,515 movies.

Our 10-fold cross-validation approach validated this hybrid synergy using the custom Hybrid Score:

- **K-Means (on 384D plot embeddings):** Achieved a strong mean score of 0.3784 ($\sigma = 0.0059$), landing firmly in the “GOOD” score level by capturing deep plot differences.
- **Hierarchical Clustering (on 19D genre matrices):** Scored a mean of 0.3572 ($\sigma = 0.0065$). While the score is slightly less than that of K-Means, it gives out the essential macro-boundaries for the clusters. Its Silhouette Coefficient of $\bar{s} = 0.13$ is entirely acceptable because of the overlap of multi-label movies.

Visuals like the Cluster–Genre Purity Heatmap and Hierarchical Genre Tree Dendrogram proved that our unsupervised models accurately constructed interpretable categories without any supervision. For an example, the Ward’s linkage successfully grouped logical pairs like Animation–Family and larger blocks like Action–Thriller–Horror–SciFi.

Finally, these components work better together. While the text embeddings discover plot similarities that simple tags miss, hierarchical genre trees hold those recommendations so they deviate away from the user’s preferences. Combined with cosine-similarity retrieval, this multi-layered framework fixes individual algorithmic flaws to give output to well rounded and accurate recommendations.

XII. LIMITATIONS

- 1) **lack of user feedback.** the system works completely on the data set features so it cannot utilize the user reviews and feedback, and consider those to give recommendations.
- 2) **English-centric model.** The `all-MiniLM-L6-v2` this model is trained on the english language, so it cannot generate embeddings of other language movies , limiting the number of movies in the recommendation system

- 3) **can’t add new genres.** The genre binariser and hierarchical cluster assignment are fixed at training time. A new genre label not present in $\mathcal{G}$ cannot be accommodated without retraining.
- 4) **can’t add new movies real time** the vector embeddings are computed offline and are not updated when movies come out. New movies require re-ingestion into ChromaDB and potentially re-clustering.
- 5) **Fixed penalty weight λ .** The scaling factor of Euclidean distance $\lambda = 0.1$ in the our Hybrid Relevancy Score was set heuristically. A limitation is that this factor is not dynamic and doesn’t adjust based on users preferences.

XIII. FUTURE WORK

- 1) Adding a multi-language model therefore increasing the scope of the number of movies that can be added to our system and would improve embedding quality for non-English movies.
- 2) Dynamic cluster assignment. as new movies are added into the our system,a dynamic clustering algorithm that integrates these movies
- 3) Dynamic relevancy Score. The Score penalty weight λ could be optimized based on the preferences of users and feedback etc

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, Hong Kong, China, Nov. 2019, pp. 3982–3992.
- [3] M. J. Pazzani and D. Billsus, “Content-based recommendation systems,” in *The Adaptive Web*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Springer, 2007, pp. 325–341.
- [4] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in *Recommender Systems Handbook*, F. Ricci et al., Eds. Springer, 2011, pp. 73–105.
- [5] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and H. Jiang, “BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer,” in *Proc. CIKM*, Beijing, China, Nov. 2019, pp. 1441–1450.
- [6] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Recommender systems for large-scale e-commerce: Scalable neighborhood formation using clustering,” in *Proc. ICCA*, Dhaka, Bangladesh, 2002.
- [7] S. Zhang, L. Yao, A. Sun, and Y. Tay, “Deep learning based recommender system: A survey and new perspectives,” *ACM Comput. Surv.*, vol. 52, no. 1, pp. 1–38, Feb. 2019.
- [8] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.
- [9] J. H. Ward Jr., “Hierarchical grouping to optimize an objective function,” *J. Amer. Statist. Assoc.*, vol. 58, no. 301, pp. 236–244, 1963.
- [10] Chroma, “Chroma: The AI-native open-source embedding database,” 2023. [Online]. Available: <https://www.trychroma.com/>
- [11] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, Nov. 1987.
- [12] A. Rosenberg and J. Hirschberg, “V-measure: A conditional entropy-based external cluster evaluation measure,” in *Proc. EMNLP-CoNLL*, Prague, Czech Republic, Jun. 2007, pp. 410–420.
- [13] P. Cremonesi, Y. Koren, and R. Turrin, “Performance of recommender algorithms on top- N recommendation tasks,” in *Proc. RecSys*, Barcelona, Spain, Sep. 2010, pp. 39–46.

- [14] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Prob.*, vol. 1, 1967, pp. 281–297.
- [15] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. ACM-SIAM SODA*, New Orleans, LA, USA, Jan. 2007, pp. 1027–1035.